



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE
Corso di Laurea triennale in Sicurezza dei Sistemi e
delle Reti Informatiche

BERTolina:
Large Language Model for
Vulnerabilities Detection on
Solidity Smart Contracts

Relatore: Chiara Braghin

Tesi di Laurea di:
Mattia Pietromartire
Matr.742294

Anno Accademico 2024 - 2025

*To Eustachio, Lea and Laura.
Thank you for always being there.*

Abstract

Smart contracts are self-executing programs deployed on the blockchain, characterized by their transparency and immutability. They cannot be modified once deployed, making the validation process essential to ensure the absence of exploitable vulnerabilities.

This thesis proposes an approach based on deep learning techniques for smart contract security analysis. The research makes use of CodeBERT, a pre-trained model specifically designed for source code analysis, in order to identify vulnerability patterns with greater accuracy than conventional tools. The model, named BERTolina, was fine-tuned on a dataset of 106.474 open-source smart contracts collected from the Ethereum blockchain, enabling it to learn the characteristics of vulnerable smart contracts.

Results demonstrate that BERTolina achieves an 89% accuracy in identifying vulnerabilities in previously unseen smart contracts, suggesting the highly potential of this approach to improve smart contract security in blockchain applications.

Contents

1	Background	5
1.1	Blockchain	5
1.1.1	Ethereum	6
1.1.2	Smart Contracts	8
1.1.3	Token and NFT	9
1.1.4	Smart Contracts Vulnerabilities	10
1.2	Machine Learning	19
2	State of the Art	21
2.1	A Typical Verification Workflow	22
2.1.1	Documentation Analysis	22
2.1.2	Static Analysis	22
2.1.3	Dynamic Analysis	23
2.1.4	Analysis Results	24
2.2	Critics	25
2.3	LLM as Detection Mechanism	26
3	Methodology	28
3.1	Architecture	28
3.2	Dataset	29
3.2.1	BERTolina: CodeBERT Fine-tuning	33
3.2.2	Evaluation Metrics	36
4	Experimental Results	42
4.1	Discussion of BERTolina results	42
4.2	Comparing BERTolina with other LLM approaches	44
5	Conclusion and future work	46
5.1	Limitations	48
5.2	Future Developments	48

Introduction

Interest for Smart Contract has constantly grown in the latest years, both in public and private sectors. Their self-execution capability, when predetermined conditions are met, makes Smart Contracts highly versatile tools applicable across numerous sectors. Among these diverse industries, several particularly stand out: decentralized finance, supply chain management, insurance, intellectual property protection, digital identity, and the entire domain of gaming and digital collectibles. [15]

The intrinsic transparency of these contracts, however, has a double side to it. While this characteristic enhances their monitorability, it simultaneously exposes them to potential attacks targeting vulnerabilities within their code. Between the most notable attacks are worth a mention:

- **The attack to The DAO (2016):** this is one of the first and best known exploit; in this case, a vulnerability in the code of Decentralized Autonomous Organization (DAO) allowed an attacker a way to withdraw a sum of about 3.6 million ETH, corresponding to about 53 million dollars. The repercussions of this event were substantial, and culminated into a hard fork of the Ethereum Blockchain and the creation of Ethereum Classic. [9]
- **The attacks to bZx (2020):** in a very short timespan, namely in less than a week, bZx platform was the target of two different attacks. The perpetrators were able to exploit a vulnerability in the protocol and to manipulate the price of assets, gaining a total profit of about 8 million dollars. [10]
- **The attack on Wormhole (2022):** a vulnerability in the code, allowed an attacker to subtract the equivalent of around 325 million of dollars in ETH, leveraging a flaw in the logic of the protocol. [39]

These examples are the witnesses of how the very high economic value managed by smart contracts, makes them a target for cyber attacks, which could

lead to considerable and unpredictable financial losses. Such incidents underline the importance of granting that smart contracts are and should be developed and validated with the highest care, to the end of minimizing all risks.

Security verification of smart contracts is currently based on human revisions, supported by instruments that are able to analyze the code and its behavior during the execution cases, locating categories of known vulnerabilities. Their accuracy can variate based on the complexity of the contract and of the specific implementations of the tools used, therefore it can not be granted a high level of accuracy.

A possible solution to this resides in the usage of automatic learning techniques (*Machine Learning*) for the analysis of vulnerabilities. In particular, the language models of highest dimensions (*Large Language Models*) have demonstrated a notable capacity of comprehension and analysis of the code; this is thanks to their ability of contextualization and identification of patterns. Large Language Models therefore, represent a promising alternative to traditional tools. [23]

This thesis aims to demonstrate the efficacy of the Large Language Models (LLM) in the identification of vulnerabilities inside smart contracts. For this purpose, it was conducted a training of the model, using a dataset consisting of 106.474 smart contracts, publicly available of the Ethereum blockchain, defined by a significant presence of vulnerabilities.

Once the training phase ended, the new model, named BERTolina, was evaluated for its capabilities in detecting vulnerabilities using smart contracts not included in the training dataset. The results show that BERTolina achieved an accuracy rate of 89% in its ability to identify code vulnerabilities.

Chapter 1

Background

In this section are introduced some essential concepts that will be addressed inside the thesis, such as Blockchain, Ethereum, Smart Contract, Machine Learning and Fine-tuning.

1.1 Blockchain

In 2008 Satoshi Nakamoto released the paper *Bitcoin: A Peer-to-Peer Electronic Cash System* [30] introducing the concept of blockchain. This was said to be a digital distributed ledger, decentralized and immutable, whose fundamental characteristics resided in the cryptographic security and the structure of the data. Indeed, the Blockchain is formed by blocks, any of which contains a series of transactions that it is identified through an hash, a cryptographic code synthesizing its content. Any alteration of the content of blocks would lead to a change on the hash, this will make immediately clear that there would have been a manipulation.

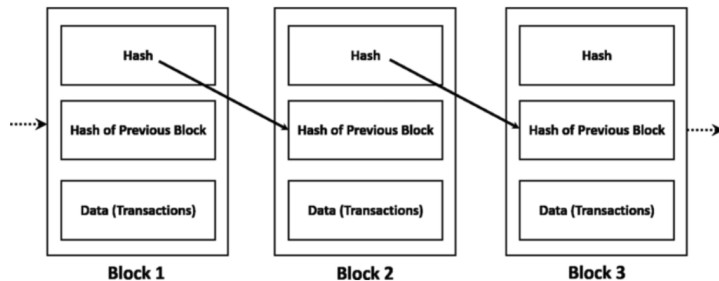


Figure 1.1: Example of Blockchain architecture [28]

In order to grant the validity and the chronological order of the transactions, the blockchain employs consensus mechanisms, such as Proof of Work (PoW) or Proof of Stake (PoS). Such mechanisms, although in different ways in their different implementations, have as their aim preventing the duplication of transactions and assuring that only the transactions that are validated are added to the blocks.

Blockchain technology use a distributed architecture where a network of computers, called *nodes*, cooperatively verify and manage information, granting a notable resistance towards attacks and attempts of censorship. This is thanks to the absence of a single point of vulnerability.

Various blockchain typologies exist, classified based on their application. Among the leading types we can find: blockchains designed for the execution of smart contracts, those optimized for payment transactions, and those focused on scalability and interoperability between networks. In fact, the introduction of the blockchain has paved the way for numerous applications, ranging from cryptocurrencies to electronic voting systems, from supply chain management to the certification of digital works of art.

In this thesis, the analysis and discussion will focus exclusively on the Ethereum blockchain. This choice is motivated by its relevance and diffusion in the field of smart contracts, as well as by its architecture and its specific characteristics.

1.1.1 Ethereum

In 2013, Vitalik Buterin proposed a blockchain platform that was innovative, open-source, decentralized and distributed. His vision was to provide developers with a tool for creating and distributing smart contracts and decentralized applications (dApps), inaugurating a new technology for the blockchain [8]. The official launch of the platform took place in 2015.

During the early stages of its life, the Ethereum blockchain adopted the consensus mechanism of Proof-of Work (PoW). This mechanism is based on *miners* which are competing for the resolution of a series of complex mathematical problems. The first miner to find the valid solution to those problems would add a new block to the blockchain, and for this it would receive as a reward Ethers (ETH), the native crypto currency of the platform. An historic moment for Ethereum took place during the September of 2022, with the transition from the PoW mechanism to that of Proof-of-Stake (PoS). This epochal change led to the disappearance of “miners”, which were substituted in their job by *validators*. The validators are entities specifically

selected to create new blocks based on the quantity of Ethers detained and bounded as warranty.

Unlike Proof of Work, where miners perform complex calculations, in PoS mechanism the validators duty is limited to validate transactions. This different methodology has a very crucial outcome: validating transactions reduces significantly the energy consumption of the Ethereum blockchain. Validators who attempt to manipulate the chain or validate fraudulent transactions will incur in the loss of the ETH they have staked, granting this way the integrity of the system. [2]

The transaction from Proof-of-Work to Proof-of-Stake ignited a heated debate inside the Ethereum community, focusing mainly on the shift from a model that was perceived as decentralize, based on the activity of the mines, to a model that could potentially favor an increased centralization. Such concern was rooted in the fact that validators who own a higher quantity of ETH, have a higher chance to be selected for the creation of a new block, with the subsequent risk to concentrate the validation power in the hand of a restricted number of entities.

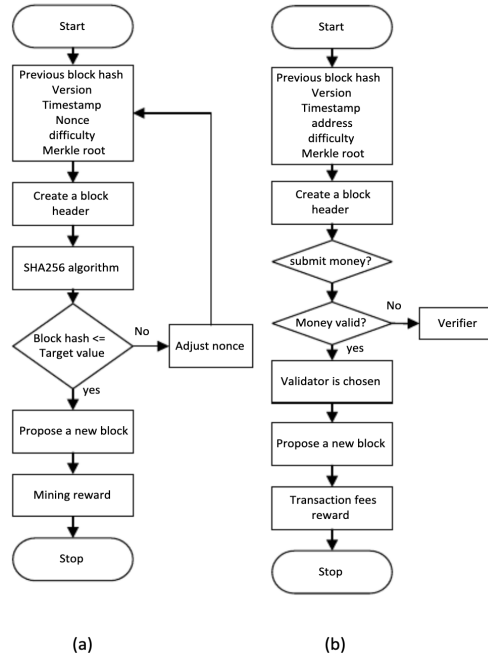


Figure 1.2: Consensus algorithms: (a)PoW, (b)PoS
[20]

The beating heart of the Ethereum architecture is constituted by the Ethereum Virtual Machine (EVM) which consists in a distributed virtual machine, responsible for the execution of the smart contracts. Operating simultaneously on all the nodes of the network, the EVM grants a uniform and predictable execution of the smart contracts, providing this way an environment of execution that is isolated and able to withstand a wide range of calculations (Turing-complete). This characteristic makes the EVM particularly suitable for the development of decentralized applications, allowing the developers to implement complex logics. [1]

1.1.2 Smart Contracts

Smart contracts are autonomous and programmable digital contracts, whose execution is distributed on the blockchain, guaranteeing their immutability and transparency, eliminating the need for intermediaries. The execution of a smart contract occurs automatically when the predefined conditions occur, without the possibility of subsequent modification. The public nature of the blockchain guarantees that the code is accessible to all participants, ensuring the complete transparency of operations.

The versatility of smart contracts favored their adoption in a vast array of application fields, such as:

- **Decentralized Finance (DeFi):** Smart contracts serve as the infrastructure for automated financial services, such as peer-to-peer lending, cryptocurrency exchanges, and digital assets management.
- **Supply Chain Management:** The use of smart contracts allows the traceability and verification of the integrity of products along the entire chain of production and distribution.
- **Healthcare Sector:** Smart contracts are used in the secure management of patients data, in the traceability of drugs and in the optimization of administrative processes.
- **Decentralized Governance:** The implementation of smart contracts facilitates the conduct of electronic voting, the management of decentralized decision-making processes and the transparent administration of public funds.

The creation of smart contracts is facilitated through the programming language Solidity, which was specifically developed for the Ethereum blockchain

and has subsequently gained adoption across various other platforms. Solidity's syntax bears a notable resemblance to JavaScript, which has significantly contributed to its widespread acceptance and popularity among developers. This syntactical familiarity has allowed developers with prior JavaScript experience to transition more easily to blockchain development, thus accelerating the adoption curve for smart contract programming. The language offers a robust set of features designed to address the unique requirements of blockchain applications, including built-in security mechanisms, contract inheritance capabilities, and efficient memory management.

Interacting with smart contracts on the Ethereum blockchain requires the payment of a fee, called *gas*, and expressed in ETH. These fees are fundamental for the network because it is useful for:

1. They regulate the demand for computational resources by ensuring that the network is not overloaded.
2. They prevent spam, discouraging the sending of unnecessary or malicious transactions.
3. They act as an incentive for the network's nodes, which carry out the activity of validating transactions. The reward, in the form of ETH, motivates validators to maintain the integrity and security of the blockchain.

1.1.3 Token and NFT

In the blockchain ecosystem, the term *Token* refers to digital assets that are fungible, that is, interchangeable with each other and have intrinsic value, such as currencies or company shares. Their fungible nature allows for their free circulation and use in various applications, from decentralized finance to payment systems.

In contrast, *Non-Fungible Tokens* (NFTs) are unique and non-interchangeable digital assets, certifying the ownership of a specific physical or digital objects. The unique digital identity of each NFT, combined with the transparency of the blockchain, allows the ownership and origin of the assets to be easily verified, giving them significant level of authenticity and traceability.

Feature	Fungible tokens	Non-fungible tokens (NFTs)
Value	Identical in value	Unique and distinct value
Interchangeability	Interchangeable with one another	Not interchangeable
Use cases	Currency, payment tokens	Collectibles, digital art, gaming
Token standards (Ethereum)	ERC-20	ERC-721, ERC-1155
Rarity	Common, can be easily replicated	Rare, hard to replicate
Example	Bitcoin, Ether, Stablecoins	CryptoKitties, digital art
Uniqueness	No unique attributes	Unique attributes and metadata
Traceability	Tracked collectively	Tracked individually

Figure 1.3: Differences between Token and Non-Fungible-Token [34]

1.1.4 Smart Contracts Vulnerabilities

The most prevalent vulnerabilities associated with smart contracts require careful examination and are elaborated upon in the following paragraphs.

Reentrancy

This vulnerability occurs when a smart contract function can be called multiple times by an external contract, before the original call has been completed. This attack mechanism is based on exploiting the operational execution order within the vulnerable contract: an attacker creates a malicious contract which, after invoking the vulnerable function in the victim contracts, uses the fallback function for recall it again, before the execution cycles has finished. This allows the attacker to execute multiple withdrawals before the balance is updated, draining all the available funds.

An example of reentrancy attack is the incident involving The DAO, one of the first decentralized autonomous organizations (DAO). In 2016, an attacker exploited a vulnerability in The DAO's code in order to execute a reentrancy attack, draining all the organization's funds, which at the time amounted to 70 million dollars. This event had significant repercussions, leading the Ethereum community to perform blockchain hard fork to recover the funds. The hard fork created a new blockchain, called Ethereum, and renamed the original blockchain, the one with the stolen funds, Ethereum Classic. [9]

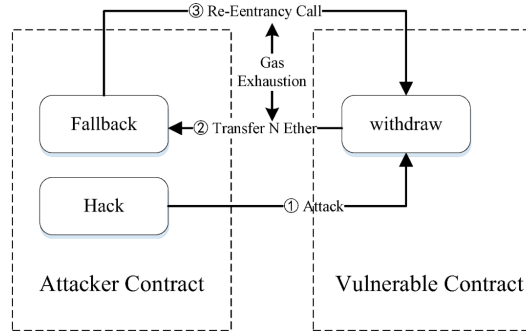


Figure 1.4: Reentrancy vulnerability principle [21]

Integer Overflow

In Ethereum, smart contracts operate with a fixed data dimension and, when an arithmetic operation exceeds the maximum capacity of the data type in use, a vulnerability known as Integer Overflow rises. In some cases, an attacker could exploit the overflow by executing malicious code inside the memory contracts. In other cases, the overflow could be used to manipulate the balance of a financial contract.

Starting from December 2020 with the release of Solidity 0.8.0, automatic checks for integer overflow and underflow were introduced. This means that when an arithmetic operation would cause an overflow, or underflow, instead of continuing with incorrect values, the contract automatically halts his execution and generates an error message.

Integer overflow is at the base of attack that took place in 2018 against Beauty Ecosystem Chain (BEC), a blockchain dedicated to the beauty ecosystem. The company's smart contract had a vulnerability in the token transfer management that allowed an attacker to exploit an integer overflow vulnerability and generate unlimited numbers of token out of nowhere. As shown in Figure 1.5, this action led to BEC devaluation, causing heavy financial losses from the investors. [38].



Figure 1.5: BEC price drop after the attack

Access Control

Access Control is a type of vulnerability that could be raised in case of incorrect access permissions management in the smart contract code. This occurs when function access control mechanisms are not implemented properly. In the presence of such vulnerabilities, an attacker could be able to invoke privileged functions and gain access to sensitive information or, even worse, acquire unauthorized privileges that could compromise the contract's integrity.

An example of access control vulnerability is represented by the accident that struck Party Wallet, an open source multi-signature wallet that users used to store their keys to Ether cryptocurrency. In 2017, an attacker discovered and exploited a flaw in the authentication mechanisms within a shared library code, used in the wallet, resulting in the compromise of three different multi-signature wallets, and a total loss of 30 million dollars in Ether.

As shown in the code below, the function *initWallet* was used to: configure the authorized owners (*_owners*), the minimum number of signs request to approve a transaction (*_required*) and the daily withdraw limit (*_daylimit*).

```
function initWallet(address[] _owners, uint _required, uint
    ↪ _daylimit) {
    initDaylimit(_daylimit);
    initMultiowned(_owners, _required);
}
```

Despite the safety measurements, there wasn't any kind of control over the fact that the wallet had been already initialized or not.

Moreover, the use of *delegatecall* in the *payable* function exposed all the library's public function to be recalled from any external users, including the *initWallet* function.

```
function() payable {
  // just being sent some cash?
  if (msg.value > 0)
    Deposit(msg.sender, msg.value);
  else if (msg.data.length > 0)
    _walletLibrary.delegatecall(msg.data);
}
```

The attacker, thanks to *payable()* function, was able to send a transaction which called *initWallet* and passing carefully constructed parameters which have setting himself as the sole owner of the wallet, modified the minimum number of signs to zero and setting extremely high daily limit withdraws. He then proceeded to send a transaction to transfer all the funds to an address he controlled.

A second accident with different consequences, occurred only a few months later. A user identified as *devops199* discovered the same vulnerability in an updated library version. After taking control of such a library, he erroneously invoked the *kill* function, causing the self-destruction of the library. Since a large number of multi-sign wallets depend on this shared library for their operations, its elimination made it impossible to access to the funds inside those wallets, resulting in a freezing of more than 500.000 ETH, which corresponded to about 150 million dollars of the time. Because of intrinsic immutability of the blockchain, became the freezing of such funds permanent, without any chance of restoration. [36]

Unchecked Low Level Calls

In the context of programming smart contracts on Ethereum, the interaction between contracts or with external addresses occurs through low-level functions, such as *call*, *delegate* and *send*. However, the use of such functions involves an intrinsic risk: in the event of a call failure, the function does not automatically stops, but returns a boolean value indicating the outcome of the operation. In the absence of adequate control over this return value, the contract may be vulnerable to unexpected and potentially harmful behaviours. In fact, an omission in the verification of such value may lead the

contract to continue its execution by mistakenly assuming that the call was successful, potentially compromising the integrity of the contract and the security of the managed funds. This vulnerability, known as unchecked low-level calls, is one of the most common and underestimated security issues in the development of smart contracts.

An example of such vulnerability is that of the case perpetrated against the protocol King of the Ether in 2016, one of the first ever games developed on the Ethereum blockchain.

The goal of the game was to become “king” and to keep that position. To be able to gain that status, the players had to send ETH to a smart contract; the one whose offer was the highest would become the new king, while the previous king would receive in exchange a part of the new offer.

However, as shown in the code below, the contract presented a critical vulnerability related to the utilization of the *msg.sender.send* methodology, which limited the quantity of available gas for the execution of the code to 2300 units. Once the available gas was finished, the withdraw could not be done and was so unsuccessful, but, the *msg.sender.send* function would keep its execution going, giving back just a “false” boolean value to report such failure. That led to a wrong calculation of the *etherLeft* variable, compromising the withdrawal function.

```
function withdraw (uint256 _amount) public
{
    require (balances [msg.sender] >= _amount
    );
    balances [msg.sender] -= _amount;
    etherLeft -= amount ;
    msg.sender.send (_amount) ;
}
```

An attacker was able to exploit this vulnerability and stole 98.5 ETH, a considerable sum for the time. Fortunately, the attacker then alerted the game developers about this vulnerability and voluntarily returned all the stolen ETH, to their rightful owners, through a manual transfer to the wallets used in the contract. [26]

Denial of Service (DoS)

Denial of Service attack uses a strategy that aims to compromise the availability of a system by generating a high workload. In the context of smart contracts, this type of attack focuses on the exhaustion of resources that

are essential for the functioning of the blockchain, such as the anomalous consumption of computational cycles, the occupation of space available for transactions or the exhaustion of gas resources, all with the aim of making the contract inaccessible to legitimate users.

Often the attacker implement these mechanisms as a preliminary phase or as a support to other malevolent intentions, deliberately creating congestion in the network to obstruct potential countermeasures. This tactic is particularly effective when the goal is to prevent the timely recovery of the stolen funds through other vulnerabilities; this is because it exploits the delay established by the congestion to complete fraudulent operations before they could be contrasted.

Bad Randomness

Generating pseudo-random numbers in smart contracts is a real difficulty, mainly due to the deterministic nature of the Ethereum blockchain. Using inadequate sources of randomness, such as *block.timestamp*, *block.number* or *block.hash*, exposes contracts to significant vulnerabilities, as these values can be influenced by miners. This issue represents a real risk to the integrity of decentralized applications that require elements of randomness in their operational processes.

An example is represented by the attack perpetrated in 2019 against the smart contract TheRun, a decentralized lottery based on Ethereum, which used *block.hash*, *block.timestamp* e *block.number* to generate in a semi-random manner the winning numbers of the lottery. As of now, the information on this attack result being very limited, but the vulnerability exploited in the attack seems to be connected to the weakness of the mechanism that generated the random numbers or to the control of the timing of the transactions.[33]

Below, by way of explaining, it is shown the venerable code of the smart contract TheRun. [4].

```
uint256 constant private salt = block.timestamp;

function random(uint Max) constant private returns (uint256
    ↪ result){
    //get the best seed for randomness
    uint256 x = salt * 100 / Max;
```

```
uint256 y = salt * block.number / (salt % 5) ;
uint256 seed = block.number/3 + (salt % 300) +
    ↪ Last_Payout +y;
uint256 h = uint256(block.blockhash(seed));

return uint256((h / x)) % Max + 1; //random number
    ↪ between 1 and Max
}
```

Lack of Input Validation

The security of smart contracts depends crucially on the validation of input data and, the lack of adequate input verification mechanisms, could expose contracts to significant vulnerabilities, allowing a potential attacker to provide specially manipulated parameters that can compromise both the operational logic of the contract and its access control mechanisms.

In 2024, the decentralized finance protocol Li.Fi was the victim of an attack of this type. Shortly after the implementation of a new smart contract, an attacker gained access to 135 wallets managed by the protocol, stealing several tokens with a total value of approximately 12 million dollars. The main vulnerability was located in the way that the contract handled input parameters in swap functions between different blockchains. In particular, the contract did not properly validate parameters related to token addresses and their values, allowing the attacker to manipulate this data to divert funds to addresses under its control. [25]

Logic Errors

Logical errors within smart contract code constitute a particularly insidious category of vulnerabilities. These vulnerabilities emerge when the contract implements flawed logic in its operations, allowing users to execute sequences of actions outside the normal execution flow. Such vulnerabilities can manifest in various functional components of the contract, compromising essential aspects such as reward calculation systems, token generation and distribution mechanisms, or the overall operational logic of the decentralized application.

A famous case of logical errors vulnerability is represented by the attack on the BNO protocol in 2023, a decentralized finance (DeFi) platform, dedicated to peer-to-peer lending, which had implemented smart contracts to automatically manage transactions and collateral mechanisms between users. The contract contained a flaw in its own calculation logic, for tokens used

as collateral, that did not adequately account for price variations under low liquidity conditions. An attacker was able to exploiting this weakness, with a series of transactions, to artificially manipulate the value of the collateral tokens, creating a discrepancy between the real value and the value recognized by the system. After inflating the collateral value, the aggressor was able to request a loans much higher than the actual collateral provided. The operation concluded with the misappropriation of funds worth approximately \$500,000.

Short Address Attack

Ethereum addresses are alphanumeric strings composed of 42 characters. The lack of adequate control on the length of the addresses given by smart contract, could trigger a vulnerability known as *Short Address Attack*, in which the attacker is able to input an address that is shorter than the 42 required characters. To have the correct length, the contract would automatically add zeros at the end of it and, given that those characters are not part of the original address, the token would be sent to an address which differs from the intended one, an address that is supposedly controlled by the attacker.

In 2017 several smaller cryptocurrency exchanges fell victim to attacks that had all the characteristics of a Short Address Attack. While the specific details of these incidents remain somewhat obscure, some research has identified this vulnerability as a possible cause of the compromise of several exchanges. By exploiting the lack of address length controls in smart contracts, attackers were able to steal large amounts of tokens.

The release of Solidity version 0.5 in 2018, represented a significant advancement in smart contract security. This version introduced improvements in the code control system during the development phase, implementing more rigorous checks that contributed to mitigating the risk associated with the Short Address Attack.

Flash Loan Attack

Flash loans are a particular type of unsecured lending in decentralized finance. Their characteristic is the possibility offered to users to access large amounts of cryptocurrencies without the need to deposit a collateral, provided that the entire amount borrowed is repaid within the same Ethereum transaction.

An attacker could exploit this condition in order to obtain flash loans, and

use the funds achieved to perform a series of operations to his advantage. These operations could include the manipulation of the markets thanks to the influence on the pool of liquidity or the exploitation of vulnerabilities identified in similar smart contracts inside the ecosystem. At the end of the operations, the attacker could proceed to the repayment of the amount that was initially borrowed. The attack relies on the nature of blockchain transactions, which ensures that all operations within a transaction either execute or fail simultaneously.

In 2021, this type of attack was used against Alpha Homora V2, a smart contract in the decentralized finance ecosystem. The attacker managed to exploit a weakness in how Alpha Homora verified the permissions for loans from Iron Bank (part of the C.R.E.A.M. Finance ecosystem), allowing them to borrow funds in amounts exceeding authorized limits. Through this vulnerability, the attacker succeeded in stealing approximately 37.5 million dollars, making it one of the most significant exploits of 2021.[27]

1.2 Machine Learning

Machine learning is a branch of artificial intelligence which focuses on the development of systems that are able to learn and improve autonomously, with no need of specific programming for every single action.

This approach is at the base of the creation of mathematical models that, through the analysis of significant quantities of data, are able to adapt and perfect themselves autonomously. In recent years, deep learning saw a huge growth in popularity; these are models that, inspired by the way the human brain works, utilize multi-layer artificial neural networks that are able to recognize intricate schemes and reach very high levels of accuracy in a wide range of applications.

Between the most advanced deep learning models we can find the Large Language Models (LLMs). They are specifically designed for the comprehension and the generation of natural language, trained on enormous text catalogs, and are based on the Transformer architecture [41], a particularly effective neural net in the elaboration of text sequences and in the learning of relationships between words and phrases.

An emblematic example of LLM is the widespread ChatGPT [35], that uses exactly the architecture to generate texts that are coherent and to entertain interactive conversations with the users, and has thus seen an exponential growth in user adoption in a very short time span.

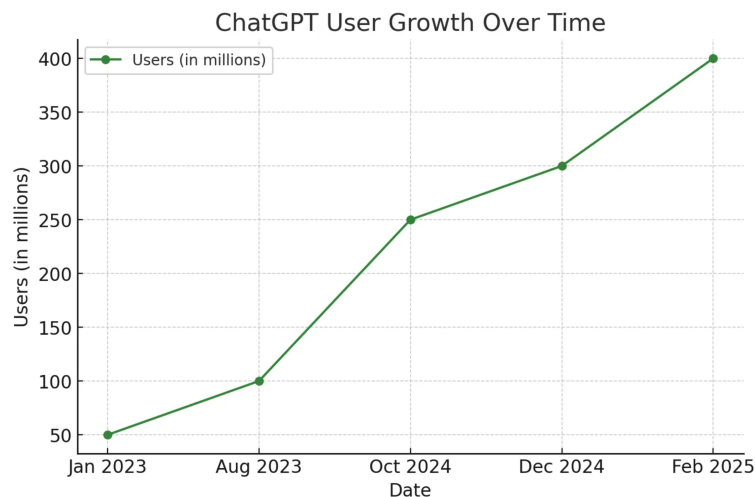


Figure 1.6: ChatGPT user adoption

The training of a Machine Learning model is based on the use of datasets, which are sets of data structured and organized appositely for the purpose of learning, and that “teach” to the models how to perform specific actions. The model learns to recognize patterns, acquiring the capability of making predictions and to undertake decisions in an autonomous way. Once the training phase is completed, a pre-trained model is obtained that is able to perform determined action in a generalized manner. To refine the learning of pre-trained models to specific tasks, *fine-tuning* needs to be performed, that is the further training with smaller and more specific datasets.

Depending on the type of pre-training and to the objective that needs to be achieved, there are different techniques of performing fine-tuning; the most common ones are the ones below:

- **Supervised Fine Tuning (SFT):** It is one of the most widespread techniques. A specific dataset is created that contains examples of desired inputs and outputs and the model learns to map them, trying to minimize the error between the generated output and the one provided. At the end of the training phase, the model is tested on evaluation data.
- **Reward Model:** It is a reward model that evaluates the quality of answers. It provides feedback to the language model indicating which answers are correct and which are not. It uses data provided by humans who express their preferences and uses automatic metrics capable of measuring the quality of the text.
- **Orthogonal Reward Policy Optimization (ORPO):** This technique rewards the model not only for providing the correct answer but also for providing it in the right way. It is used to train models to provide answers that align with those preferred by humans.
- **Generative Policy Optimization (GPO):** Technique that focuses on the ability to choose the right words at the right time to generate coherent and relevant text. It focuses on optimizing the LLM policy.

Chapter 2

State of the Art

Identifying vulnerabilities in smart contracts is a crucial process for the security and reliability of decentralized applications. Currently, this task is entrusted to highly specialized personnel who use a combination of tools and methodologies. Specifically, the typical exam involves the use of static and dynamic analysis tools, which examine the source code and the behaviour of the contract during execution, followed by manual audits conducted by security experts, which analyse the code for potential vulnerabilities.

In recent years, the landscape of tools for identifying vulnerabilities in smart contracts has undergone a significant evolution. The traditional approach, based on static analysis, has seen the emergence of tools such as *Slither*, *Mythril* and *Securify*, which analyse the source code or bytecode in search of known vulnerable patterns.

Dynamic analysis has also seen significant progresses. Tools such as *Oyente* and *Manticore*, run code in controlled environments to detect anomalous behaviour. The combination of symbolic execution and fuzzing techniques has significantly improved the effectiveness of these approaches, allowing them to discover vulnerabilities that could otherwise be missed by static analysis.

The most innovative approach is represented by the application of machine learning and deep learning techniques which, through tools such as *VulnHunt-GPT* and similar ones, are demonstrating promising results, thanks to the ability to automatically learn complex patterns associated with vulnerabilities.

2.1 A Typical Verification Workflow

Below it is proposed a detailed analysis of the workflow typically adopted to identify vulnerabilities in smart contracts.

2.1.1 Documentation Analysis

The process begins with the analysis of the technical documentation, an important preliminary phase, which allows to understand the functionalities and logic of the smart contract under analysis. During this phase, the main functionalities, execution flows, and any exchange of information with other contracts or with the outside world, are identified.

After the preliminary identification of potential problems, the experts perform an in-depth analysis of the possible threats and vulnerabilities. This phase is particularly delicate and requires detailed knowledge not only of the different types of vulnerabilities, but also of their severity and the specific risks they involved in the context of the smart contract under examination. Auditors must carefully evaluate which parts of the code could be subject to attacks and what consequences could arise, considering factors such as the economic impact, the probability of exploitation and possible mitigation strategies.

2.1.2 Static Analysis

The next stage is the static analysis of the source code. During this stage, automated tools examine the code without executing it, looking for patterns that may be related to known vulnerabilities.

Typically, analysts make use of one or more of the following tools:

- **Slither:** A framework capable of performing an extremely detailed analysis, identifying more than 70 types of vulnerabilities with different levels of complexity and criticality. For each vulnerability it generates a report that includes the description of the vulnerability, its location in the code, the potential security risks and, finally, proposes mitigation actions. [12]

- **Mythril:** A tool capable of combining symbolic analysis and fuzzing techniques to identify potential vulnerabilities in smart contracts. It is able to analyse codes written in Solidity and machine codes executable by the Ethereum Virtual Machine. Mythril can explore all possible execution paths, generating inputs that can activate anomalous behaviours and is also able to generate complex attack scenarios. It is a tool suitable for detecting vulnerabilities such as Integer Overflow and Underflow, Reentrancy, Access Control and manipulations of the execution flow. [11]
- **Securify:** An innovative static analysis code tool that operates through customizable security policies, defined by the auditor. It analyses the source code, comparing it with predefined standards, then it generates detailed and structured feedbacks, offering developers indications for the mitigation of the identified risks. [7]
- **SmartCheck:** Tool that transforms Solidity code into XML so that the potential security risks can be more effectively analysed. The tool can examine XML code identifying recurring patterns of vulnerabilities, such as Integer Overflow and Underflow, Race Condition, Reentrancy, Access Control. [40]

```
(mythril) cerebral-cortex:contracts bernhardmueller$ myth -x Ownable.sol
==== Unprotected Selfdestruct ====
SWC ID: 106
Severity: High
Contract: Ownable
Function name: kill()
PC address: 251
Estimated Gas Usage: 619 - 1044
The contract can be killed by anyone.
Anyone can kill this contract and withdraw its balance to an arbitrary address.
-----
In file: Ownable.sol:13

selfdestruct(msg.sender)
-----
```

Figure 2.1: Example of Mythril interface [14]

2.1.3 Dynamic Analysis

Once the static analysis phase is completed, the process moves forward to the test and simulation phase, where the smart contracts are executed in a controlled environment to observe their behaviours. The goal is to verify the behaviour of the system and whether emerge anomalous or unexpected behaviours that could be exploited by potential attackers. The most commonly adopted tools in this context include:

- **Oyente:** One of the first and most important tools for Ethereum smart contract dynamic analysis. It works by converting the bytecode into an intermediate representation, which is then analysed to identify potential problematic execution paths. It was specifically designed to detect four types of vulnerabilities: Reentrancy, Bad Randomness, transaction execution order issues, and issues related to improper exception handling. [18]
- **Manticore:** A dynamic analysis tool, specifically designed for examining EVM bytecode. Its unique feature is the ability to perform in-depth tests without the need for a real blockchain, operating in a simulated environment that faithfully reproduces the code execution conditions. During the analysis, Manticore builds mathematical representations of the contract state and simulates different operating conditions, automatically identifying anomalies such as arithmetic overflows, reentrancy problems or logical inconsistencies. [29]
- **Harvey:** A tool that adopts an approach based on the generation of random or semi-random inputs (fuzzing) that are used as input into the contract under examination. Its operation is based on the ability to automatically produce thousands of diversified test cases, simultaneously monitoring the behaviour of the contract in search of anomalous behaviours, unexpected crashes or error conditions that could signal the presence of vulnerabilities.[42]
- **Maian:** A tool that operates directly on the bytecode of smart contracts, particularly effective to identify vulnerabilities at the execution level. It examines the behaviour of the contract through a series of consecutive transactions, analysing how the state of the contract evolves over time. This tool was specifically designed to detect three basic categories of vulnerabilities: "suicide" contracts, contracts that can lock up funds inside them indefinitely, and contracts that allow unauthorized transfers of Ether to arbitrary addresses. [32]

2.1.4 Analysis Results

In the final phase, the evidence of the previous activities (documentation, static and dynamic analysis) are aggregated, interpreted and documented in a systematic way.

It starts with the unification of the reports generated by tools like Slither, Mythril, Oyente, and Manticore, each of which may have identified different

portant to recognize their limitations.

First, static analysis tools, which are needed to quickly identify potential vulnerabilities, rely on a predefined set of rules that can lead to a high number of false positives, i.e. alerts that report potential vulnerabilities that do not exist. The need to manually verify each alert involves a significant waste of time and resources, slowing down the analysis process. Additionally, these tools struggle to analyse interactions with data external to the blockchain, limiting their effectiveness in contracts that rely on oracles or other off-chain data sources.

On the other hand, dynamic analysis tools require long execution times and are not able to test all possible combinations of contract states, due to the exponential growth of combinations during execution. The reliability of the dynamic analysis results is also linked to the accuracy of the test environment. An inaccurate or non-representative environment can lead to false negatives, or the failure to identify real vulnerabilities.

Finally, it is important to underline that both static and dynamic analysis tools are based on the identification of known patterns, derived from previously discovered vulnerabilities. This based approach makes them unable to detect zero-day vulnerabilities, i.e. vulnerabilities unknown at the time of analysis, which represent a significant threat to the security of smart contracts.

2.3 LLM as Detection Mechanism

In this context, the emergence of LLMs opens up new perspectives in the security analysis of smart contracts. In recent years, several studies have explored the use of machine learning and deep learning techniques to automate and improve the process of finding potential weaknesses in a code.

For example, in 2021, the researchers Ashizawa, Yanai, Cruz, and Okamurac proposed Eth2Vec, a machine learning model that uses vector representations of code to identify vulnerabilities in Ethereum smart contracts. As described in their article, *Eth2Vec: Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts* [5], the model uses a natural language processing neural network to automatically learn the characteristics of vulnerable EVM bytecode, thus overcoming the need to manually define the features to extract.

More recently, Boi, Esposito and Lee, developed VulnHunt-GPT, a system that leverages the capabilities of GPT-3 to identify vulnerabilities in smart

contracts. In their paper *VulnHunt-GPT: a Smart Contract vulnerabilities detector based on OpenAI ChatGPT* [6], the authors demonstrate how large language models can be effectively adapted for this specific task. The system is capable of detecting a wide range of vulnerabilities, including reentrancy, access control, and bad randomness issues, while also providing suggestions for their mitigation.

Both of these approaches represent a step forward compared to traditional methods, as they are capable of learning complex patterns and adapting to new types of vulnerabilities. The integration of machine learning techniques with the ability to understand the source code, offers a promising prospect for improving the security of smart contracts in the blockchain ecosystem.

Chapter 3

Methodology

This chapter describes the architecture of the model used, the dataset and data preparation, and the procedure used to train the model.

3.1 Architecture

The nature of the data allows us to address the problem of identifying vulnerabilities using Natural Language Processing (NLP) techniques, which are techniques that allow computers to understand the meaning of human language [24]. It's not just about recognizing words, but interpreting their meaning, the intentions behind them, and the context in which they are used. These techniques use the Transformer [41] architecture, which instead of considering the input word by word, considers all the words at once, giving greater attention to those important words, necessary to understand the meaning of the entire sentence. This is possible thanks to "weights" that indicate the importance of a word within the sentence and the context. These weights are learned during the training phase. The Transformer architecture is designed with an encoder and a decoder. The encoder has the purpose of providing a numerical representation of the input sentence, so that it can be processed by the model, while the decoder has the purpose of transforming the series of numbers it receives into text understandable for humans.

There are planned encoder-only models, such as BERT (Bidirectional Encoder Representations from Transformers) [13]. Proposed by Devlin in 2018, it is particularly effective for tasks that require analysing and understanding the meaning of input text. In fact, it was trained on a very large dataset and is capable of understanding 3.3 billion words.

In 2020, Microsoft introduced CodeBERT [17], a fine tuning of the BERT model, capable of understanding the syntax of programming languages such

as Python, PHP, Ruby, GO, C#, C++, Java and JavaScript. The similarity of Solidity’s syntax to that of JavaScript led me to choose CodeBERT as a pre-trained model on which to perform a fine tuning on vulnerable and non-vulnerable smart contracts.

3.2 Dataset

For the fine-tuning of CodeBERT model, it was chosen to utilized the dataset *Slither-Audited-Smart-Contracts* [37], publicly available on the HuggingFace platform [22], a well-known reference point for machine learning resources. It contains 106.474 smart contracts, divided into training-set (84,859 contracts), validation-set (10,608 contracts) and test-set (10,607 contracts). This dataset was chosen because of its widespread use in smart contract vulnerability analysis research. The dataset was built starting from the list of smart contracts present in Smart Contract Sanctuary and the source code was downloaded from that repository or, alternatively, through Etherscan [16]. Then, every contract was analysed with the static analysis tool Slither, which had found and classified 38 different kind of potential vulnerabilities, expressed in specific Slither labels. Table 3.1 shows the complete mapping from the 38 Slither specific labels to the broader categories used in this study. For example, the Slither labels ’suicidal’, which refers to the detection of the now deprecated *selfdestruct* Solidity function, and ’arbitrary-send’ were both grouped into the broader category ’access-control’.

Etichetta Slither	Categoria Vulnerabilità
suicidal	access-control
arbitrary-send	access-control
unused-return	unchecked-calls
unchecked-lowlevel	unchecked-calls
erc20-interface	ignore
msg-value-loop	double-spending
controlled-array-length	access-control
locked-ether	locked-ether
shadowing-state	ignore
tx-origin	access-control
delegatecall-loop	double-spending
shadowing-abstract	ignore
constant-function-state	other
divide-before-multiply	arithmetic

array-by-reference	other
incorrect-equality	other
name-reused	ignore
incorrect-shift	ignore
rtlo	other
backdoor	other
write-after-write	ignore
weak-prng	bad-randomness
public-mappings-nested	ignore
controlled-delegatecall	access-control
reentrancy-no-eth	reentrancy
constant-function-asm	other
reused-constructor	ignore
reentrancy-eth	reentrancy
unprotected-upgrade	access-control
uninitialized-storage	other
tautology	ignore
unchecked-send	locked-ether
erc721-interface	ignore
uninitialized-local	other
unchecked-transfer	unchecked-calls
boolean-cst	ignore
mapping-deletion	other
uninitialized-state	other
safe	safe

Table 3.1: Slither Categories

The dataset has two different labelling modes: one in textual format, where the vulnerabilities are described in words, and one “multilabel”, which encodes them numerically. Since CodeBERT is a classification model that works better with numeric inputs, it was chosen to use the numeric version, called “all-multilabel”. Each set is therefore composed of:

- **Address:** the address of the smart contract on the Ethereum network
- **Source Code:** the source code of the smart contract
- **Bytecode:** the EVM of the smart contract
- **Slither:** the output of the Slither analysis, containing the identified vulnerability categories and the related numerical indices

In this thesis, it was chosen to focus exclusively on the source code, therefore the information related to addresses and bytecode were eliminated from the dataset, proceeding to train the model only using the source code and the corresponding vulnerability classes.

It was also chosen not to remove the comments present in the smart contract code in order to provide context to the CodeBERT model.

Below, an example of the Python code used to select and extract the *Source Code* and *Slither* columns from the original dataset, creating a new simplified dataset containing only this information.

```
#Carico il dataset da HuggingFace
dataset = load_dataset("mwratescode/slither-audited-smart-
    ↪ contracts", "big-multilabel")

#Creo un dizionario con source_code e categoria
    ↪ vulnerabilit 
def preprocess_data(example):
    results = []
    for i in range(len(example['address'])):
        results.append({"text": example["source_code"][i], "
            ↪ labels": example["slither"][i]})
    return {"text": [x["text"] for x in results], "labels": [
        ↪ x["labels"] for x in results]}

#Riscrivo il dataset
dataset = dataset.map(preprocess_data, batched=True,
    ↪ remove_columns=dataset.column_names['train'])
dataset = dataset.flatten()

#Test del dataset rielaborato
print(dataset["test"][:1])
```

The pre-processed dataset is then passed to the tokenizer, a key component that transforms the text into numbers, the format in which the model can actually work. CodeBERT uses a tokenizer called WordPiece, which is part of the family of "subword tokenizers" used widely in Transformer models. Its peculiarity is that it doesn't just break the text into whole words, but divides it into smaller units, the "subwords". These units can be common words, parts of words (such as prefixes or suffixes), or even single characters.

This method is particularly useful for code because it allows you to better manage rare words or technical terms, such as names of variables or functions, and to recognize similarities between words with the same root. CodeBERT's

source_code string	slither list
pragma solidity 0.5.4; interface IERC20 { function balanceOf(address account) external view returns (uint256); function transfer(address recipient, uint256 amount) external returns (bool); function_	[4]
pragma solidity 0.6.10; pragma experimental ABIEncoderV2; library BokkyPooBahsDateTimelibrary { uint constant SECONDS_PER_DAY = 24 * 60 * 60; uint constant SECONDS_PER_HOUR = 60 * 60; uint constant_	[6,4,9,7]
pragma solidity 0.6.5; pragma experimental ABIEncoderV2; struct UserInfo { uint256 amount; uint256 rewardDebt; } struct PoolInfo { address lpToken; uint256 allocPoint; uint256 lastRewardBlock; uint256_	[38]
pragma solidity 0.6.6; library Address { /** * @dev Returns true if 'account' is a contract. * * [IMPORTANT] * ===== It is unsafe to assume that an address for which this function returns * false is a_	[38]
pragma solidity 0.6.8; pragma experimental ABIEncoderV2; contract Initializable { /** * @dev Indicates that the contract has been initialized. */ bool private initialized; /** * @dev Indicates that the_	[0,4,7,9,16,5,2]
pragma solidity 0.4.26; interface IERC20 { function totalSupply() external view returns (uint256); function balanceOf(address who) external view returns (uint256); function allowance(address owner,_	[2]

Figure 3.1: Refined Dataset

tokenizer has a "mixed" vocabulary, because it was trained on a large set of texts in both natural language and programming languages, which makes it suitable for understanding both natural language that may be present in comments (e.g. 'the', 'a', 'is') and specific words in programming code (e.g. 'function', 'require', ';', '{').

Once the conversion is complete, the tokenizer applies the attention mask, another key element for models like CodeBERT. In practice, a binary mask is applied to each token of the input sequence, thereby indicating which part is important and which can be ignored. This is because Transformers-based models work better with inputs of uniform length and, when processing variable-length batches, such as smart contracts, the tokenizer adds padding tokens. These padding tokens, however, do not contain any useful information but only serve to reach the correct size. The attention mask marks the padding tokens as 'not important', allowing CodeBERT to focus only on the source code.

The final step in data preparation is to convert the numeric tokens and attention mask into PyTorch tensors. This transformation is also essential because tensors are the fundamental data structure that allows CodeBERT to process information efficiently and take full advantage of the GPU processing power.

3.2.1 BERTolina: CodeBERT Fine-tuning

The objective of this dissertation work is to develop a model that is able to identify vulnerabilities in smart contracts. To reach this goal, it was implemented BERTolina, a model that is capable of finding and classifying the vulnerabilities of the Solidity code. BERTolina was obtained by fine-tuning a pre-trained model called CodeBERT, that was made available by Microsoft through the platform Hugging Face. The entire process of fine-tuning was realized making use of the computational power of GPU A100 to accelerate the training of the model.

The used approach focuses on a multi-label classification: for every smart contract, BERTolina has to establish not only if a vulnerability is present or not, but, in case it is, it has to identify which vulnerabilities are found between the ones considered in the dataset.

The architecture of the resulting model was obtained by modifying CodeBERT slightly. In particular, it was used the `AutoModelForSequenceClassification` class of the Transformers library provided by Hugging Face, and was configured for a problem of multi-label classification. This automatically added a level of linear classification above the CodeBERT pre-trained model.

```
model = AutoModelForSequenceClassification.from_pretrained(  
    "microsoft/codebert-base",  
    num_labels=len(labels_list),  
    problem_type="multi_label_classification"  
)
```

The fine-tuning that was performed is of the supervised type (SFT), a methodology that requires instructing the model through tangible examples. The wide dataset labeled Slither-Audited-Smart-Contracts is fundamental, that is because it provides the necessary “supervision”. To measure and guide the learning was used the loss function Focal Loss. This function acts as a “thermometer” quantifying the error between the expectations of BERTolina and the correct labels, and directing the model toward the optimal solution. During the training, at any stage, the weight of BERTolina is progressively modified based on this Focal Loss function and on the confrontation with the labels of the training set.

```
#Focal Loss per bilanciare classi  
class FocalLoss(torch.nn.Module):  
    def __init__(self, gamma=2.0, reduction="mean"):
```

```

        super(FocalLoss, self).__init__()
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits, targets):
        bce_loss = F.binary_cross_entropy_with_logits(logits,
            ↪ targets, reduction="none")
        pt = torch.exp(-bce_loss)
        focal_loss = (1 - pt) ** self.gamma * bce_loss

        if self.reduction == "mean":
            return focal_loss.mean()
        elif self.reduction == "sum":
            return focal_loss.sum()
        else:
            return focal_loss

loss_fn = FocalLoss()

```

```

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    logits = logits.astype(np.float32)  # Evita problemi di
    ↪ casting

    threshold = dyn_thresh.compute_threshold(logits)
    preds = (logits > threshold).astype(int)

    precision, recall, f1, _ =
    ↪ precision_recall_fscore_support(labels, preds,
    ↪ average="macro")
    accuracy = (preds == labels).mean()

    return {
        "accuracy": accuracy,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }

```

During the training process of BERTolina, it was given particular attention to the selection of the hyperparameters, which are crucial elements that highly influence the quality of learning and the computational efficiency. After numerous preliminary experiments, it was reached an optimal configuration that balances predictive accuracy and resource consumptions.

The learning rate indicates how fast the model learns from its errors during the training. In this case it was chosen to have a learning rate of $2e-5$,

that is 0.00002, which represents an efficient compromise between stability of learning and speed. It is a very small figure, which allows the model to learn at a stable pace but that is not too slow.

The batch size indicated the number of examples that the model elaborated at the same time before updating its own parameters. Big batches require more GPU memory, but generate updates that are more stable; small batches require less memory but can cause updates that are less accurate.

The batch size chosen was 16 examples; this size results in being a good compromise, allowing to use the GPU in an efficient way without having to sacrifice the stability of the training.

During the training of a model, one of the most common problems is the overfitting, which occurs when the model “memorizes” the training data instead of learning the general patterns. This leads to not working correctly when new data is presented. A technique to avoid this to happen consists in establishing weight decays, that means that it is added a penalizing function, which pushes the model to use smaller weights, less prone to the problem of memorization. A value of weight decay that is too big could prevent the model to learning enough data, while a value that is too small could be not enough to prevent overfitting. An optimal value of 0.01 for the weight decay coefficient was identified through a series of systematic experiments; that is a good value that penalizes the weight of big dimensions and pushes the model towards simpler solutions.

The training was structured in complete cycles, called epoch, through the dataset. At the end of each epoch the model is assessed through the techniques that were previously cited, in order to evaluate the progress. By repeating the cycle of training multiple times, the model refines the comprehension and the identification of vulnerabilities, reaching an optimal convergence. For BERTolina it was chosen to utilize a maximum limit of 25 epochs, by setting an Early Stopping to the value of 2. This means that, if for 2 consecutive epochs the metric of evaluation (F1) does not improve, the training is automatically halted.

```
training_args = TrainingArguments(  
    output_dir="./results",  
    evaluation_strategy="epoch",  
    save_strategy="epoch",  
    learning_rate=2e-5,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=16,  
    num_train_epochs=25,  
    weight_decay=0.01,  
    load_best_model_at_end=True,
```

```

metric_for_best_model="f1",
greater_is_better=True,
logging_dir="./logs",
logging_steps=10,
fp16=True,
)

```

Finally, to maximize the efficiency of the process of training without compromising the quality of the end result, the standard functionality of the Trainer class of the Hugging Face library was extended, allowing the implementation of the mixed precision training technique. This approach uses numeric representation at 16 but (FP16) for the most part of the mathematical operations, reducing substantially the memory requisites and incrementing the computing speed on a latest generation GPU equipped with tensor accelerators. Critical operations for the numeric stability are kept instead in a standard precision (FP32), granting the integrity of the optimization process.

```

trainer = CustomTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience
        ↪ =2)],
)

```

This combination of optimized parameters, regularization techniques and computational optimizations has made possible the efficient training of BERTolina, reaching a satisfying balance between preventive accuracy, computational efficiency and generalization ability on data that was never observed during the learning phase.

3.2.2 Evaluation Metrics

Before presenting the experimental results that were obtained, it is necessary to clarify the evaluation metrics that were used to measure the efficacy of the model, which were previously mentioned. These metrics represent fundamental tools to translate the behaviour of the classifier into numeric values that are understandable and comparable.

At the base of the used metrics there are four measurements:

- **True Positive (TP):** it represents the number of cases in which the model has correctly identified an instance as belonging to the class of interest. In our specific context, it indicated the number of actual vulnerabilities that were correctly detected.
- **True Negative (TN):** it indicates the number of instances that were correctly classified as non belonging to the target class. It refers to the code segments that are free from vulnerabilities that the model has correctly recognized as secure.
- **False Positive (FP):** it indicated the number of overestimation errors, which is the case in which the model has erroneously classified in an instance as belonging to a class to which it actually didn't belong. These false alarms represent secure code that was mistakenly flagged as vulnerable.
- **False Negative (FN):** it measures the underestimation error, that is the situation in which the model was not able to recognise an instance belonging to the target class. In our case, it represents actual vulnerabilities that were missed by the detection system, and it is potentially the most critical error in the context of cyber security.

These four measurements constitute the foundation on which all the other more complex metrics that will be used to evaluate BERTolina performances are based. It will be thus shown how these measurements are combined to obtain more relevant indicators such as precision, recall, F1-score and accuracy, each of which concentrate on a different aspect regarding the efficacy of the implemented system.

Precision

As previously mentioned, Precision represents the metric that evaluates the reliability of the positive forecasts performed by the model. Such metric is calculated through a mathematical formula that relates the number of True Positives (TP) with the sum of True Positives and False Positives (FP).

$$Precision = \frac{TP}{TP + FP}$$

In the case of a multi-class classification, Precision can be calculated through different ways:

- **MicroPrecision:** it aggregates inputs from all the classes, calculating the total of True Positives and False Positives, to then determine the relation. Every instance is assigned with the same importance, and is thus more influenced by the classes that are more frequent.

$$MicroPrecision = \frac{\sum TP}{\sum TP + \sum FP}$$

This metric is particularly useful when it is desired to give more credit to the classes with a greater number of instances, or when the distribution of classes is significantly unbalanced.

- **MacroPrecision:** it calculated separately the precision of each class, and then determined the arithmetic mean. It treats all the classes with the same importance, independently of their frequency in the data.

$$MacroPrecision = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i + FP_i}$$

This metric is particularly suitable if it is desired to evaluate the performance of the model in a balanced way on every class, even the least represented.

- **WeightedPrecision:** it represents a compromise between the previous. An average of the specific precisions per class is calculated, and then weighted based on the frequency of each class.

$$WeightedPrecision = \sum_{i=1}^n w_i \cdot \frac{TP_i}{TP_i + FP_i}$$

This metric is particularly useful when it is desired to take into account the distribution of the classes, without completely neglecting the minority classes, as it could happen with the microprecision.

Recall

Unlike precision, which measures the reliability of positive predictions, recall quantifies the model's ability to identify all positive examples present in the dataset. It is expressed as the ratio of the number of true positives (TP) to the sum of true positives and false negatives (FN).

$$Recall = \frac{TP}{TP + FN}$$

It is of crucial importance especially in those applicative areas in which the cost associated to False Negatives is particularly high, as in the case of cyber security, where every False Negative corresponds to a vulnerability that was not detected and could lead to being exploited from attacker, with potentially devastating consequences for the security of the system.

In the multi-class scope, the recall could also be calculated through different methodologies, each of which provides complementary information of the performances of the model.

- **MicroRecall:** it aggregates the contributions from all the classes, calculating the total sum of True Positives and False Positives through all the classes in order to determine their ratio. A proportional importance is attributed to the frequency of classes, resulting in being particularly influenced by the majority classes.

$$MicroRecall = \frac{\sum TP}{\sum TP + \sum FN}$$

It is preferable when it is wanted to give greater weight to the most common types of vulnerabilities.

- **MacroRecall:** it separately calculated the recall for each class and subsequently determined their arithmetic mean. It treats all the classes as equal in importance, independently from their frequency.

$$MacroRecall = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i + FN_i}$$

It is indicated when it is wanted to equally evaluate the capacity of the model on individuation the different typologies of vulnerabilities, even the rarest, but potentially critical, ones.

- **WeightedRecall:** it represents the compromise between the previous two. A mean of specific recall for each class is calculated, and this

weighted based on the relative frequency of the classes.

$$WeightedRecall = \sum_{i=1}^n w_i \cdot \frac{TP_i}{TP_i + FN_i}$$

F1-Score

F1-Score arises from the need to find a compromise between Precision and Recall, and it represents one of the most complete and balanced metric to evaluate the performance of a classification model. F1-Score could be interpreted as the harmonic mean of Precision and Recall, providing this way a single figure that efficiently synthesizes both the aspects. In addition, the harmonic mean tends to penalize more the imbalances, favouring models that maintain high values in both the metrics instead of excelling in just one.

$$F1Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Also in this case, when dealing with multi-classes, F1-Score can be calculated in different ways:

- **Micro F1:** it is calculated by aggregating the contributions from all the classes and determining a single value of Precision and Recall, from which the F1- Score originates.

$$MicroF1 = 2 \cdot \frac{MicroPrecision \cdot MicroRecall}{MicroPrecision + MicroRecall}$$

MicroF1 results being particularly influenced by the most frequent classes and represent a suitable choice when it is wanted to give more weight to the majority classes.

- **MacroF1:** it separately calculates the F1-Score for each class and then determines the arithmetic mean. It treats all the classes as equal in importance, independently from their frequency.

$$MacroF1 = \frac{1}{n} \sum_{i=1}^n F1_i$$

This metric is particularly suitable when it is desired to evaluate the performance of the model in an equal way on all the classes, including the minority ones.

- **WeightedF1:** it represents a compromise between MicroF1 and MacroF1. A mean between the F1-Scores of each class is calculated, and then weighted based on the relative frequency of the classes.

$$WeightedF1 = \sum_{i=1}^n w_i \cdot F1_i$$

This metric offers a good compromise between MicroF1 and MacroF1, taking into account the natural distribution of the classes, without completely neglecting the minority ones.

Accuracy

Accuracy represents one of the most intuitive metrics to evaluate the performances of a classification model. It measures the proportion of correct predictions compared to the total number of predictions performed, so it is nothing more than the ratio between the number of correct predictions and the number of total predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

It provides a percentage value included between 0 and 100, which quantifies the correct forecasts performed by the model. It is therefore an easily understandable metric and simple to calculate.

In a multi-class context, it maintains its formulation as the ratio between correct forecasts and total forecast, and, in the presence of datasets that are characterized by a highly imbalanced distribution between the different classes, it can provide a misleading vision of the performances of the model. Therefore, Accuracy is normally accompanied by other metrics of complementary evaluation, such as Precision, Recall and F1-Score. This integrated approach allows to obtain a characterization that is better exhaustive and balanced between the performances of the model.

Chapter 4

Experimental Results

In this chapter, will be illustrated and analyzed the results obtained by the BERTolina model during the evaluation phase.

The analysis was conducted utilizing the validation split of the dataset, consisting in near 11.000 smart contracts not used during the training phase. Such contracts represent a wide variety of use-cases and structural complexity, allowing to verify this way the efficiency of the model in different scenarios. The evaluation metrics presented in the following section offer a complete vision of the performances of BERTolina, underling the strengths but also the limitations that were encountered during the analysis.

4.1 Discussion of BERTolina results

The results of the fine-tuning process of the model are presented in detail in Table 4.1. Although the training was initially configured to pursue up to 25 epochs, it was interrupted in advance at the 21st epoch. This premature arrest was determined by the implementation of a mechanism of early stopping, activated after experiencing an exigent phenomenon of overfitting: in fact, for two consecutive epochs the model didn't show significant improvements in the performance on the validation set, indicating a decreased capacity for generalization and an excessive adaptation to the training data.

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.375500	0.337933	84.40%	66.06%	73.52%	61.48%
2	0.333900	0.314019	86.12%	69.79%	78.97%	64.29%
3	0.233200	0.307004	87.02%	72.50%	78.24%	68.38%
4	0.273500	0.301931	87.57%	73.43%	80.92%	68.56%
5	0.225100	0.289463	88.25%	75.13%	81.48%	70.16%
6	0.189900	0.291072	88.71%	76.29%	82.43%	71.42%
7	0.215500	0.306638	88.77%	76.47%	82.30%	71.72%
8	0.152800	0.314127	89.03%	77.04%	82.71%	72.29%
9	0.153400	0.319319	89.04%	77.04%	83.19%	72.09%
10	0.172800	0.316908	89.31%	77.61%	83.39%	72.81%
11	0.159700	0.334423	89.27%	77.71%	83.01%	73.16%
12	0.163000	0.340034	89.53%	78.13%	83.73%	73.42%
13	0.131400	0.352396	89.41%	77.76%	83.80%	72.91%
14	0.119500	0.356023	89.55%	78.19%	83.77%	73.48%
15	0.105800	0.368381	89.55%	78.32%	83.88%	73.60%
16	0.130600	0.376553	89.60%	78.28%	83.67%	73.69%
17	0.122100	0.388485	89.65%	78.51%	83.76%	73.97%
18	0.118400	0.394911	89.70%	78.54%	84.16%	73.79%
19	0.125300	0.402235	89.78%	78.67%	84.17%	73.99%
20	0.100600	0.411493	89.97%	79.03%	84.70%	74.26%
21	0.105800	0.417936	89.99%	78.88%	84.55%	74.08%
22	0.107900	0.420763	89.88%	78.87%	84.44%	74.15%

Table 4.1: BERTolina: Validation Metrics (in percentage)

Analyzing the loss functions *Training Loss* and *Validation Loss*, which quantify the errors committed by the model, it is possible to observe a divergent behaviour between the training set and the validation set. The loss on the training set shows a trend that is overall decreasing, going from an initial value of 0,3755 on the first epoch down to 0,1079 on the twenty-second epoch, with some in-between oscillations. This trend highlights the progressive ability of the model of adapting to training data.

Differently, the loss on the validation set presents a behaviour initially decreasing down to reaching the minimal value of 0,2895 on epoch 5, to then begin a phase of constant growth up to 0,4208 at epoch 22. This divergence between the two trends is a clear signal of overfitting: the model keeps continuously improving its performances on training data, but progressively lose the ability of generalization on the data not seen in the validation set.

In contrast with the growth of the loss in the validation set, the performance metrics show a constant improvement during the entire training process. Accuracy gradually grows from 84,40% on epoch 1, up to 89,88% on epoch 22, with an overall incrementation of 5,48 percentage points. Similarly, the F1-Score grows from 66,07% to 78,87%, Precision from 73,52% to 84,45%, and Recall from 61,48% to 74,15%

This means that, despite the growing overfitting, the model is still able to improve its capabilities of selection between classes. The loss function, in fact, measures when the model is safe in its forecasts, while performance metrics evaluate the accuracy of the classification that are actually generated. In other words, the model progressively becomes less calibrated but more accurate in the final classifications.

Moreover, analyzing the metrics in detail, it is possible to observe that Precision grows faster than Recall during the training phase. This suggests that the model improves in its ability of avoiding False Positives, while it progresses in a slower way when it comes to its ability of identifying all the existing positive cases.

In conclusion, the valuation conducted on the validation set, highlighted the ability of BERTolina to identify vulnerabilities inside smart contracts, reaching meaningful results with an F1-Score of 79,03% and Accuracy of 89,97%. Such metrics, considered in the context of the intrinsic complexity of automated code analysis, suggest that the proposed model represents an effective approach to the problem, laying a solid foundation for further developments in this research area.

4.2 Comparing BERTolina with other LLM approaches

In the evaluation of BERTolina, it is important to perform a comparison with other recent approaches that have used LLM to identify vulnerabilities in smart contracts.

Although VulnHunt-GPT shares the ultimate goal with BERTolina, the comparison is impractical for two main reasons:

1. **Difference in evaluation metrics:** As said, BERTolina’s evaluation is based on standard classification metrics such as Accuracy, Precision,

Recall and F1-score that measure the ability of the model to classify smart contracts. In contrast, VulnHunt-GPT does not provide these metrics but instead focuses on the number of vulnerabilities identified and the total identification and execution time, making a direct comparison impossible.

2. **Difference in datasets:** There is a significant difference in the number of smart contracts used to train the models. VulnHunt-GPT reports using 69 smart contracts, whereas BERTolina was trained on over 10,000.

For these reasons, it is not possible to make a direct comparison between the two models.

Eth2Vec, on the other hand, despite being trained on a different dataset and operating on vector analysis derived from EVM bytecode rather than Solidity, provides these metrics for each vulnerability category.

Vulnerability	Severity	Eth2Vec		
		Precision	Recall	F1-score
Reentrancy	3	86.6	54.8	61.5
Time dependency	2	75.2	17.0	27.3
ERC-20 transfer	1	95.6	58.4	72.4
Gas consumption	1	48.0	29.0	32.4
Implicit visibility	1	68.9	82.0	74.8
Integer overflow	1	89.9	57.6	70.1
Integer underflow	1	74.6	56.0	63.7
Average		77.0	50.7	57.5
Standard deviation		14.7	19.7	18.0

Figure 4.1: Eth2Vec Vulnerability detection results

By examining the average of each metrics, it is possible to make an indicative comparison between the two models.

Eth2Vec achieves an average F1-score of 57,5%, a Precision of 77% and a Recall of 50,7%. Comparing these values with the averages obtained by BERTolina on the validation set (i.e., F1-score 79.03%, Precision 84,7% and Recall 74,26%), it is evident that BERTolina shows higher average values, particularly for F1-Score and Recall.

However, this data must be interpreted taking into account the differences in approach and dataset used.

Chapter 5

Conclusion and future work

In this thesis work I addressed the problem of automatic identification of vulnerabilities in smart contracts using deep learning techniques.

Starting from an in-depth analysis of smart contracts, of the fundamental mechanism of the blockchain (Proof of Work and Proof of Stake) and of their most widespread applications (such as NFTs and tokens), it was highlighted how security represents one of the most critical challenges for this emerging technology. Vulnerabilities in smart contracts, due to the intrinsic immutability of the blockchain, could in fact lead to particularly serious consequences, as demonstrated by numerous cases of attacks that resulted in significant financial losses.

During my research I explored the potential of machine learning and, in particular, of deep learning in addressing this problem. The use of advanced architectures such as Transformers, thanks to self-attention mechanisms, to the vast ability of comprehension and also to the tokenization (fundamental process for transforming source code into an appropriate input), represents the starting point from which the presented solution was built.

Relying on these theoretical foundations it was developed BERTolina, a model based on Code-BERT specifically fine-tuned for the analysis of smart contracts written in Solidity. The objective was to create a tool that was able to automatically identify vulnerabilities in the code, contributing this way to the security of the blockchain ecosystem. The results that were obtained highlighted encouraging performances. Indeed BERTolina reached a level of accuracy of 89,88% and an F1-Score of 79% on validation datasets, demonstrating a good capacity of discrimination between vulnerable contracts and non-vulnerable contracts. Particularly significant was the precision value,

which reached 84.45%, indicating a high reliability of the model in correctly classifying the contracts as vulnerable. The recall, although reaching slightly lower values (74.15%), remains an appreciable result considering the complexity of the application domain.

Through the analysis of the trend of the metrics during the fine-tuning process, some interesting characteristics of the model's behavior were observed. It emerged in particular a clear phenomenon of overfitting after the first training epochs, highlighted by the divergence between the training loss and the validation loss. Despite this phenomenon, the performance metrics kept improving, suggesting that the model, although losing the ability to generalise in probabilistic terms, maintained a good efficacy in binary classification. Another aspect observed during the training phase is the tendency of the model of privileging Precision over Recall. This behaviour made BERTolina progressively more conservative in its positive predictions, reducing the False Positives at the expense of a lower capacity of identifying the entirety of the positive cases. Such a characteristic could be considered an appropriate choice in context where the cost of a False Negative is potentially much higher than that of a False Positive.

From a methodological point of view, the adoption of CodeBERT as a pre-trained model and its subsequent adaptation to the specific domain of smart contracts, proved to be an effective choice. This strategy made it possible to exploit the previous knowledge on programming languages already incorporated in the model, perfecting it for the specificities of the Solidity language. Representing the code as a sequence of tokens, paying attention to the syntax elements that are most important, helped the model in individuating recurrent schemes associated with the vulnerabilities. Such a strategy made it possible to optimize both the development timing and the overall performance of the system.

In a nutshell, the results that were obtained confirm the validity of the approach that is suggested and position BERTolina as a promising tool in the automatic analysis of smart contracts, contributing to the solution ecosystem to improve the security of blockchain applications.

5.1 Limitations

While recognizing the encouraging results obtained, it is essential to consider some limitations of the present work for a complete evaluation of its contributions.

- **Overfitting:** as highlighted from the analysis of the loss curves, the model showed a tendency to overfitting after the first epoch of training. Although the implementation of early stopping mitigated partially this problem, a discrepancy between the performance on the training set and the one on the validation set is represented, indicating a generalization capacity that is not optimal.
- **Precision-Recall Balance:** the model shows a preference toward Precision over Recall. This characteristic, although desirable in some application contexts, represents a limit in the ability of the system in identifying all the existing vulnerabilities.
- **Computational Complexity:** The use of a complex architecture such as CodeBERT implies a significant computational cost, both in training and inference phases, which may limit the applicability of the system in contexts with limited resources or in necessity of real time analytics.

5.2 Future Developments

The proposed thesis work opens up numerous possibilities for future developments, both from a methodological and applicative point of view.

One of the first improvements that needs to be fulfilled is related to the optimization of BERTolina performances. During the model’s development were, in fact, used standard training parameters that, although producing satisfying results, still leave margins for improvements. The incrementation of the batch size dimension during the training phase is something that needs to be taken into consideration. In the present work, a relatively small batch size was used, an increase in this parameter could lead to greater stability guaranteeing more accurate estimates and the general improvement of the model. This adjustment would require far more substantial computational resources, but the potential benefits in terms of quality of the model and reduction of training time could justify the investment.

To also address the overfitting problem, it could be useful to implement more

sophisticated regularization techniques to be added to the early stopping already in use. Approaches of the dropout type, the regularization L1/L2 with optimised parameters or specific data augmentation techniques for the source code, are possible pathways to be explored.

Concerning the imbalance between Precision and Recall, it would be interesting to implement an adjustable classification threshold that allowed to prefer Precision in the context where False Positives are particularly costly or vice versa, to maximize Recall when it is important not to miss any potential vulnerability. In this way BERTolina could be used both during the developing phase of a smart contract, and during the phase of audit pre-deployment.

Regarding the dataset, it could be useful to optimize the balance of the distribution of vulnerability classes, as well as enriching it including emerging vulnerability or less common vulnerability examples.

A promising further development could be the integration of the approach used by Eth2Vec, which is based on EVM opcode analysis to generate vector representations. This method could complement BERTolina's current source code based analysis, offering the possibility to identify vulnerabilities even when the code is not available and to detect low level vulnerabilities.

Finally, with the evolution of the blockchain, new languages for the smart contracts are emerging. It would thus be interesting to explore the adaptability to languages different from Solidity, such as Vyper, Rust.

This future development represents only a few of the possible directions toward which the present work could be extended. The fast paced evolution of both the blockchain technologies and of machine learning techniques, constantly provide new instruments and approaches that could be integrated in future versions of BERTolina, opening the path to solutions that will be more and more efficient for the identification of vulnerabilities inside the code.

Bibliography

- [1] Ethereum Virtual Machine (EVM). URL: <https://ethereum.org/en/developers/docs/evm/>.
- [2] Proof-of-stake (PoS). URL: <https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/>.
- [3] Smart Contract Weakness Classification (SWC). Accesso: Marzo 2025. URL: <https://swcregistry.io/>.
- [4] theRun.sol. Accesso: Marzo 2025. URL: https://github.com/crytic/not-so-smart-contracts/blob/master/bad_randomness/theRun_source_code/theRun.sol.
- [5] Nami Ashizawa, Naoto Yanai, Jason Paul Cruz, and Shingo Okamura. Eth2Vec: Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts, 2021. URL: <https://arxiv.org/abs/2101.02377>, arXiv:2101.02377.
- [6] Biagio Boi, Christian Esposito, and Sokjoon Lee. VulnHunt-GPT: a Smart Contract vulnerabilities detector based on OpenAI chatGPT, 2024. doi:10.1145/3605098.3636003.
- [7] Lexi Brent, Aleksandar Jurisevic, Bernard Kong, Eric Liu, Karl Rasmussen, and Michael Scholten. Vandal: A Scalable Security Analysis Framework for Smart Contracts. *arXiv preprint arXiv:1806.01143*, 2018. URL: <https://arxiv.org/pdf/1806.01143>.
- [8] Vitalik Buterin. Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. URL: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf.
- [9] Chainlink. Reentrancy Attacks and The DAO Hack. URL: <https://blog.chain.link/reentrancy-attacks-and-the-dao-hack/>.

- [10] Coinbase. Analysis on the bZx attack, DeFi vulnerabilities and the state of debit cards in crypto. URL: <https://www.coinbase.com/it/learn/market-updates/around-the-block-issue-3>.
- [11] ConsenSys. Mythril: Security Analysis Tool for Ethereum Smart Contracts. <https://github.com/ConsenSys/mythril>, 2025. Accessed: 27 February 2025.
- [12] Crytic. Slither: A Static Analysis Framework for Smart Contracts. <https://github.com/ConsenSys/mythril>, 2025. Accessed: 27 February 2025.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL: <https://arxiv.org/abs/1810.04805>, arXiv: 1810.04805.
- [14] Consensys diligence. All Ethereum Security Tools. URL: <https://diligence.consensys.io/blog/2019/03/all-ethereum-security-tools/>.
- [15] Econstor. Adoption and diffusion of blockchain technology. URL: <https://www.econstor.eu/bitstream/10419/289452/1/1884854117.pdf>.
- [16] Etherscan Team. Etherscan: The Ethereum Blockchain Explorer. Web Application, 2025. A comprehensive blockchain explorer and analytics platform for Ethereum. URL: <https://etherscan.io/>.
- [17] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A Pre-Trained Model for Programming and Natural Languages, 2020. URL: <https://arxiv.org/abs/2002.08155>, arXiv: 2002.08155.
- [18] Enzyme Finance. Oyente: An Analysis Tool for Smart Contracts. <https://github.com/enzymefinance/oyente>, 2023.
- [19] OWASP Foundation. Smart Contract Security Verification Standard, 2023. Accesso: Marzo 2025. URL: <https://scs.owasp.org/>.
- [20] Nedim Hajdarbegovic. Consensus algorithms: a) PoW, b) PoS, and c) PoA. ResearchGate, 2021. Figure 2 in "Blockchain and Distributed Ledger Technologies Applications". URL: <https://www>.

- researchgate.net/figure/Consensus-algorithms-a-PoW-b-PoS-and-c-PoA_fig2_351438968, doi:10.13140/RG.2.2.14343.11689/2.
- [21] Yaqiong He, Hanjie Dong, Huaiguang Wu, and Qianheng Duan. Formal Analysis of Reentrancy Vulnerabilities in Smart Contract Based on CPN. *Electronics*, 12(10), 2023. URL: <https://www.mdpi.com/2079-9292/12/10/2152>, doi:10.3390/electronics12102152.
 - [22] Hugging Face Team. Hugging Face: The Home of Machine Learning. Web Platform, 2025. A collaborative platform for the machine learning community to build, discover and share ML models and datasets. URL: <https://huggingface.co/>.
 - [23] Shi Qiu Muhammad Saqib Saeed Anwar Muhammad Usman Naveed Akhtar Nick Barnes Ajmal Mian Humza Naveed, Asad Ullah Khan. A Comprehensive Overview of Large Language Models. *arXiv:2307.06435*, 2023. URL: <https://arxiv.org/html/2307.06435v105>, arXiv:2307.06435.
 - [24] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Pearson, Various Editions - Check the one you used. URL: <https://web.stanford.edu/~jurafsky/slp3/>.
 - [25] Marko Jurina. Security Incident Report 16th July. URL: <https://li.fi/knowledge-hub/incident-report-16th-july/>.
 - [26] KotET. Post-Mortem Investigation. URL: <https://www.kingoftheether.com/thrones/kingoftheether/index.html>.
 - [27] Alpha Finance Lab. Alpha Homora V2 Post Mortem. URL: <https://blog.alphaventuredao.io/alpha-homora-v2-post-mortem/>.
 - [28] Senthil Kumar Lakshmanan. Blockchain Architecture. ResearchGate, 2023. Figure 1 in "Introduction to Blockchain Technology and Its Architecture". URL: https://www.researchgate.net/figure/Blockchain-Architecture_fig1_370692053, doi:10.13140/RG.2.2.13958.75844/1.
 - [29] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts, 2019. URL: <https://arxiv.org/abs/1907.03890>, arXiv:1907.03890.

- [30] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. URL: <https://bitcoin.org/bitcoin.pdf>.
- [31] Ivica Nikolic. Maian. URL: <https://github.com/ivicanikolicsg/MAIAN/blob/master/maian.png>.
- [32] Ivica Nikolic, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. Finding The Greedy, Prodigal, and Suicidal Contracts at Scale, 2018. URL: <https://arxiv.org/abs/1802.06038>, arXiv:1802.06038.
- [33] Hrishi Olickel. Why Smart Contracts Fail: Undiscovered bugs and what we can do about them. URL: <https://hrishiolickel.medium.com/why-smart-contracts-fail-undiscovered-bugs-and-what-we-can-do-about-them-119aa2843007>.
- [34] Irem Onder and Horst Treiblmaier. Transforming Tourism and Hospitality: The Innovative Potential of Non-Fungible Tokens (NFTs), 12 2023. doi:10.4324/9781003351917-5.
- [35] OpenAI. ChatGPT: Advanced Conversational AI Platform, 2024. URL: <https://chat.openai.com>.
- [36] Santiago Palladino. The Parity Wallet Hack Reloaded. URL: <https://blog.openzeppelin.com/parity-wallet-hack-reloaded>.
- [37] Martina Rossini. Slither Audited Smart Contracts Dataset, 2022. URL: <https://huggingface.co/datasets/mwritescode/slither-audited-smart-contracts>.
- [38] SECBIT-MEDIA. A disastrous vulnerability found in smart contracts of BeautyChain (BEC). URL: <https://medium.com/secbit-media/a-disastrous-vulnerability-found-in-smart-contracts-of-beautychain-bec-dbf24ddbc30e>.
- [39] Chainalysis Team. Wormhole Hack: February 2022. URL: <https://www.chainalysis.com/blog/wormhole-hack-february-2022/>.
- [40] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. SmartCheck: Static Analysis of Ethereum Smart Contracts. In *2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 9–16, 2018.

- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, 2023. URL: <https://arxiv.org/abs/1706.03762>, arXiv:1706.03762.
- [42] Valentin Wüstholtz and Maria Christakis. Harvey: A Greybox Fuzzer for Smart Contracts, 2019. URL: <https://arxiv.org/abs/1905.06944>, arXiv:1905.06944.